JavaScript object basics

Previous Overview: Dynamic scripting with JavaScript Next ➡

In this article, we look at how to implement JavaScript object syntax, and revisit some JavaScript syntax we've seen in earlier articles, to see how it looks in object-oriented terms.

Prerequisites: Basic computer literacy, a basic understanding of HTML and CSS, JavaScript basics as covered in previous lessons.

Learning objectives: Understand what is JavaScript object syntax and how it looks, and how it differs from other syntaxes. Learn how to use JavaScript object syntax to make your own basic JavaScript objects.

In this article
Object basics
Dot notation
Bracket notation
Setting object members
What is "this"?
Introducing constructors
Trying out some object-oriented code
Summary

Object basics

An object is a collection of related data and/or functionality. These usually consist of several variables and functions (which are sometimes called properties and methods when they are inside objects). Let's look through an example to understand what they look like.

Make a copy of the latest version of our oscillator1 file from last time, rename it to oscillator2, and add the CSS as well as the source code into it. We'll use this as a basis for our object-oriented code. Note that we've also added a new line of code to the end of the JavaScript, to log a variable called myOscillator to the console, and made it a type of variable that contains an object.

As with many things in JavaScript, creating an object often begins with defining and initializing a variable. Try entering the following into the console, and enter myOscillator to see what it returns, then, after trying out the following:

```
myOscillator.type = 'square';
myOscillator.frequency.value = 440;
myOscillator.connect(ctx.destination);
myOscillator.start(ctx.currentTime);
```

```
myOscillator.stop(ctx.currentTime + 2);
```

```
myOscillator.disconnect();
```

```
myOscillator = null;
```

```
myOscillator = ctx.createOscillator();
myOscillator.type = 'sawtooth';
myOscillator.frequency.value = 880;
myOscillator.connect(ctx.destination);
myOscillator.start(ctx.currentTime);
myOscillator.stop(ctx.currentTime + 2);
```

```
myOscillator = null;
```

```
myOscillator = ctx.createOscillator();
myOscillator.type = 'triangle';
myOscillator.frequency.value = 220;
myOscillator.connect(ctx.destination);
myOscillator.start(ctx.currentTime);
myOscillator.stop(ctx.currentTime + 2);
```

```
myOscillator = null;
```

```
myOscillator = ctx.createOscillator();
myOscillator.type = 'sine';
myOscillator.frequency.value = 110;
myOscillator.connect(ctx.destination);
myOscillator.start(ctx.currentTime);
myOscillator.stop(ctx.currentTime + 2);
```